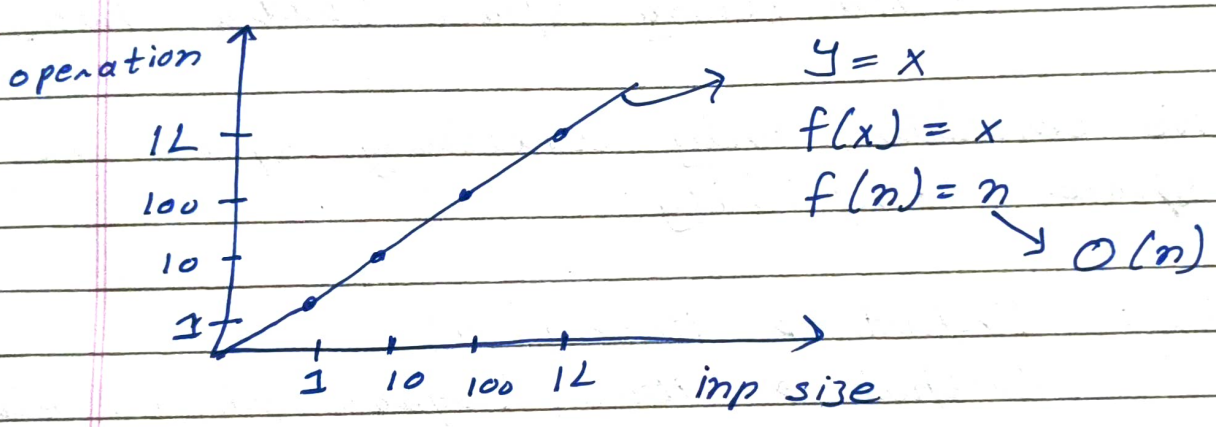
☆ Time and Space Complexity

* Time complexity:

Not the actual time but amount of time taken as function of input size (n)

Ex: Linear search

For n input → max n operations



1 Big O Notation: Worst case (Upper bound)

Step 1: Ignore constants

Step 2: Largest term

$f(n) = 4n^2 + 3n + 5$

$\Rightarrow n^2 + n + 1$

$\Rightarrow O(n^2)$

Worst Case — Avg Case — Best Case

O — θ (thetha) — Ω (omega)

Upper bound — Lower bound

* Space Complexity :

Amount of space taken by an algo. as fnc of input size (n).

Code → Input (~~the~~ Array, String ...)

→ Auxiliary Space (Extra Space)

We consider only auxiliary space in space complexity

Ex: Input → Array [n]

Output → Array (Square of all elem)

Space com. → only of output

O(n)

Ex 2: Inp → Arr [n]

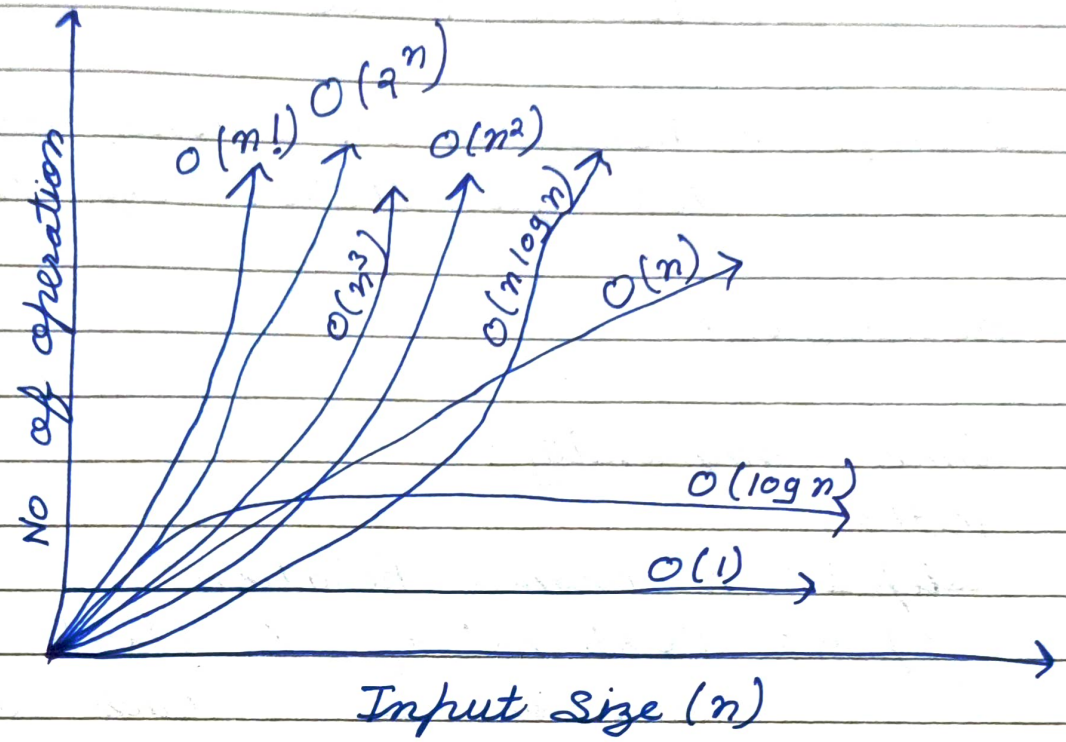
out → sum

space com → O (k) {k → constant}

→ O (1)

PAGE NO. :
DATE : / /

* Common Time complexity :



$O(1)$ → Sum of number from 1 to N
1st elem of array.

$O(n)$ → Factorial, Fibonacci
Linear Search

$O(n^2)$ → Bubble sort
Pattern Problems (mostly)

$O(n^3)$ → Sub-arrays of an array
Has 3 loops (nested)

$O(\log n)$ → Binary Search
Binary Search Tree

$O(n \log n)$ → Merge Sort, Greedy algos,
Quick sort (Avg θ)

$O(2^n)$ → Recursion
Brute Force

$O(n!)$ → Recursion (n Queens, Knight Tour)
All permutation of string.

* Recursion:

→ Time complexity:
method
~~Step~~ 1: Recurrence Relation
~~Step~~ 2: Total no of recursion call X Work in each call

→ Space Complexity

SC = Depth of call stack X Memory in each call

⇒ Height of call stack X Memory in each call